SECURITY · DOMAIN 7 - ENSURING DATA SECURITY AND COMPLIANCE

PII, anonymisation and purging

Section 7. Four methods, one blueprint, and two retention numbers that are not the same.

SECTION 7 of 10 WEIGHT 10% TOPIC Privacy

admin@sthithapragna.com

Pseudonymised data is **still personal data**. Anonymised data is not. That distinction decides what the law asks of you.

admin@sthithapragna.com

Two pairs, not a list of four

	PSEUDONYMISATION / ANONYMISATION
Scope	Record level / The whole dataset
Reversible	Yes, by authorised users / No, irreversibly altered
Still personal data	Yes / No
Methods	Hashing, tokenisation / Suppression, generalisation

ON THE EXAM

The exam guide names all four methods. Knowing which pair each belongs to answers most questions before you read the options.

Hashing and tokenisation compared

- **Hashing** transforms a value irreversibly. The same input always gives the same output, so records stay **linkable**.
- That linkability is why it is still pseudonymisation. Without a salt, a dictionary attack can recover common values.
- **Tokenisation** replaces the value with a token, and a **separate vault** maps back. Only authorised users can re-identify.
- Spark SQL also offers native encryption functions where a reversible transform is wanted without an external vault.

> ON THE EXAM
>
> Hashing is irreversible and still not anonymisation, because the same person always hashes to the same value and can be tracked across a dataset.

Suppression and generalisation

- **Suppression** excludes the identifying columns entirely. The simplest method, and the one that loses most signal.
- **Generalisation** replaces a value with a broader category, keeping some analytic value while removing identifiability.
- It also covers **removing rows where a group is too small** to be non-identifying.
- Truncating an address to a wider range still supports geographic analysis without identifying anybody.

> ON THE EXAM
>
> Generalisation is the interesting one. It preserves the signal you actually wanted while removing the part that identifies a person.

The documentation is unambiguous. **Complete deletion is preferable to obfuscation**, because eliminating re-identification risk usually requires deleting the data.

Bronze first, driven by a control table

- Databricks recommends **deleting in the bronze layer first**, then propagating to silver and gold.
- Driven by a **scheduled job that queries a table of deletion requests**. You insert a row per request.
- Use **DELETE** for a single record. Use **MERGE against the control table** when deleting many at once.
- After the merge succeeds, **update the control table** to record that the request has been processed.

> ON THE EXAM
>
> The control table is what makes the process auditable. It is the evidence that a request arrived, and that it was actioned.

Sever the link and you delete less

- Using a **non-natural key** such as a user identifier, rather than a natural key like an email address, separates PII from the rest.
- You can then **delete the personal data and leave the non-personal data in place**.
- Behavioural analysis survives, because the events remain and only the identity is gone.
- Use the email as the join key everywhere and every table containing it becomes a table you must purge.

> ON THE EXAM
>
> This is a modelling decision made years before the first deletion request, and it determines how expensive every one of them will be.

Delete is not enough on its own

- With deletion vectors enabled, a delete **marks** rows. The data remains in the underlying files.
- You must also run **REORG TABLE with the purge option** to rewrite the files without those rows.
- That applies to Delta tables, and also to **materialized views and streaming tables**.
- Module 21 introduced this. Here it becomes a compliance requirement rather than a housekeeping one.

> ON THE EXAM
>
> A right-to-erasure request is not satisfied when the DELETE succeeds. It is satisfied after the purge, and after the old files are removed.

Two retention numbers, and they are different

- Delta retains **table history, including deleted records, for 30 days** by default, for time travel and rollback.
- **VACUUM by default reduces time travel to 7 days**, and removes those historical versions from cloud storage.
- Two different properties. A question can turn on which one it is describing.
- And note: even when previous versions are removed from the table, **the data is still in cloud storage** until vacuum runs.

> ON THE EXAM
>
> Thirty days of history and a seven-day vacuum window are not in conflict. They govern different things.

Three routes, in the order the docs recommend

- **Predictive optimization** for Unity Catalog managed tables. The recommended route, and it maintains streaming tables and materialized views.
- Failing that, **Lakeflow pipelines perform maintenance automatically within 24 hours** of a streaming table or materialized view being updated.
- Failing both, run **VACUUM** yourself on Delta tables.
- Doing none of the three means deleted records remain recoverable in cloud storage indefinitely.

> ON THE EXAM
>
> Module 20's automatic maintenance and this compliance requirement are the same mechanism. Predictive optimization is doing privacy work on your behalf.

Materialized views handle it themselves

- **Materialized views automatically handle deletions in their sources.** Nothing special is required.
- You must still **refresh** the view and run maintenance for the deletion to be fully processed.
- A materialized view always returns the correct result, using incremental computation **only when it is cheaper**.
- So a deletion in a source may force a **full recompute**. Correctness is never traded for speed.

> ON THE EXAM
>
> This is module 5's 'keeps a query's answer correct' arriving as a compliance property. Deletions propagate because correctness demands it.

Streaming tables need three deliberate steps

- Streaming tables process **append-only** data. Deleting from a streaming source is **not supported and breaks the stream**.
- The documented method: **delete in the source** with DML, **delete from the streaming table** with DML, then set **skipChangeCommits** on the read.
- The order matters. You apply the deletion to both tables yourself, and only then tell the stream to ignore the change commits.
- Setting the flag **without** the two deletions would simply discard the change and leave the data present.

> ON THE EXAM
>
> Module 21 called this flag a trap. Here it is the documented answer, and the difference is whether you deleted from both tables first.

Full refresh, and what it can cost

- You can instead delete from the source and then **fully refresh** the streaming table.
- That clears the streaming state and reprocesses everything from the source.
- **Any upstream source beyond its retention period will not be reprocessed**, which causes data loss.
- Recommended **only** where historical data is still available and reprocessing is not costly.

> ON THE EXAM
>
> Module 5's retention arithmetic, appearing for the fourth time. A Kafka topic ageing out after seven days cannot replay a year.

Stream from the change feed instead

- The documentation calls this the **more robust** implementation for streaming.
- Stream from the **change feeds** of the Delta tables and handle updates and deletes in your processing code.
- That removes the need for both the skip flag and the full refresh.
- Module 21 made this argument from the performance side. The privacy documentation makes it from the compliance side.

> ON THE EXAM
>
> Two modules, two motivations, the same recommendation. That is usually a sign it is the right design.

Upstream sources are in scope too

- **The regulations apply to all data**, including sources outside Delta such as message buses, files and databases.
- Deleting inside Databricks is not sufficient. You must delete in the upstream sources as well.
- Otherwise the next ingestion run simply brings the record back.
- You may also need to **export workspace data** for compliance or backup before running deletion workflows.

> ON THE EXAM
>
> A deletion that the next pipeline run undoes is not a deletion. Scope the purge to the whole path, not just the lakehouse.

What to carry into the exam

- **Pseudonymisation is still personal data**: hashing and tokenisation. Anonymisation is not: suppression and generalisation.
- **Complete deletion is preferable to obfuscation.** Bronze first, driven by a control table.
- With deletion vectors, **DELETE is not enough**. Reorg with purge, then maintenance.
- **Materialized views handle deletions automatically. Streaming tables need three deliberate steps.**
- History defaults to **30 days**; vacuum reduces time travel to **7**. Delete upstream too.

> ON THE EXAM
>
> That completes Section 7 at 10%. Section 8, Data Governance, is next at 7%.

> A team hashes customer email addresses with a standard hash function before loading them into an analytics catalog. They tell the privacy office the data is now anonymised and outside the scope of the regulation.
>
> Analysts routinely count distinct hashed values to measure unique customers.

What is wrong with that claim?

A Nothing. Hashing is irreversible, so the data is anonymised.

B Hashing is pseudonymisation. The same input always hashes identically, so records remain linkable and it is still personal data.

C The claim holds only if the hash is salted, which makes it anonymisation.

D Hashing is anonymisation, but counting distinct values re-identifies the data.

◆ PAUSE AND DECIDE

What is wrong with that claim?

A Nothing. Hashing is irreversible, so the data is anonymised.

Irreversible is not the test. Linkability is, and hashing preserves it.

B Hashing is pseudonymisation. The same input always hashes identically, so records remain linkable and it is still personal data.

The very behaviour analysts rely on for counting unique customers is what makes it pseudonymisation.

C The claim holds only if the hash is salted, which makes it anonymisation.

A salt frustrates dictionary attacks. It does not remove linkability within the dataset.

D Hashing is anonymisation, but counting distinct values re-identifies the data.

Gets the classification wrong, then invents a mechanism to explain it.

ON THE EXAM

The analysts' own usage proves the point. If you can count unique customers, the records are still linked to individuals.

> A right-to-erasure request is processed. The engineer runs DELETE on every table containing the user's records, confirms zero rows are returned by follow-up queries, and closes the ticket.
>
> The tables have deletion vectors enabled, and the estate does not use predictive optimization.

What remains to be done?

A Nothing. The queries confirm the records are gone.

B Run REORG TABLE with the purge option, then maintenance or VACUUM, since the rows are only marked and the files still exist.

C Re-run the DELETE with a longer retention setting.

D Convert the tables to materialized views, which delete permanently.

◆ PAUSE AND DECIDE

What remains to be done?

A Nothing. The queries confirm the records are gone.

Queries confirm the rows are filtered out. They say nothing about the files.

B Run REORG TABLE with the purge option, then maintenance or VACUUM, since the rows are only marked and the files still exist.

Deletion vectors mark rather than rewrite, and history plus cloud storage retain the data until purge and vacuum.

C Re-run the DELETE with a longer retention setting.

Retention governs how long files survive, and re-running the delete changes nothing.

D Convert the tables to materialized views, which delete permanently.

Materialized views also need the purge with deletion vectors enabled.

ON THE EXAM

Zero rows returned is exactly what a deletion vector produces. The evidence the engineer collected cannot distinguish marked from removed.

> A pipeline has a bronze streaming table, a silver streaming table, and two gold materialized views. A deletion request is processed by deleting from the source Delta tables only.
>
> The next pipeline run fails on the streaming tables. The materialized views are fine.

What explains both outcomes, and what is the fix?

A Materialized views handle source deletions automatically. Streaming tables need the rows deleted from them too, then skipChangeCommits on the read.

B Streaming tables need a longer watermark to tolerate deletions.

C Materialized views ignore deletions, which is why they did not fail. Both need a full refresh.

D The deletion should have been applied to the gold layer first, then propagated upstream.

◆ PAUSE AND DECIDE

What explains both outcomes, and what is the fix?

A. Materialized views handle source deletions automatically. Streaming tables need the rows deleted from them too, then skipChangeCommits on the read.

Both halves are documented: MVs handle deletions, and streaming tables break because a delete is not an append.

B. Streaming tables need a longer watermark to tolerate deletions.

Watermarks govern late-arriving event-time data, not deletes.

C. Materialized views ignore deletions, which is why they did not fail. Both need a full refresh.

They do not ignore deletions; they process them correctly, which is why nothing broke.

D. The deletion should have been applied to the gold layer first, then propagated upstream.

The recommendation is bronze first, propagating downstream.

ON THE EXAM

The two dataset types genuinely behave differently here, and the scenario shows both behaviours in the same pipeline.

> An auditor asks how quickly deleted personal data becomes unrecoverable. An engineer answers that Delta retains history for 30 days, so it takes a month.
>
> The tables are Unity Catalog managed with predictive optimization enabled, and the default vacuum retention has not been changed.

What is the more accurate answer?

A 30 days is correct, since that is the table history retention.

B History retention and vacuum retention are different. Vacuum removes files beyond the 7-day default, and predictive optimization runs it.

C The data is unrecoverable immediately after DELETE.

D It is indefinite, because cloud storage never removes the files.

◆ PAUSE AND DECIDE

What is the more accurate answer?

A 30 days is correct, since that is the table history retention.

That governs time travel history, not how long the underlying files survive.

B History retention and vacuum retention are different. Vacuum removes files beyond the 7-day default, and predictive optimization runs it.

Two distinct properties. Vacuum's default reduces time travel to 7 days and removes the files from cloud storage.

C The data is unrecoverable immediately after DELETE.

With deletion vectors it is not even rewritten, let alone removed.

D It is indefinite, because cloud storage never removes the files.

True only if nothing ever runs maintenance. Here predictive optimization does.

ON THE EXAM

Two numbers, two properties. Confusing them gives an auditor an answer that is wrong in the direction of over-promising.

> A company processes deletion requests carefully across every Delta table in its lakehouse. Three weeks later, a routine check finds records for several deleted users present again in the bronze layer.
>
> Ingestion is from a Kafka topic with 30-day retention, replayed after an unrelated incident.

What went wrong?

A The deletion was never committed, so the records were never removed.

B The regulations apply to upstream sources too. The records were never deleted from Kafka, so a replay reintroduced them.

C Time travel restored the deleted versions automatically.

D Predictive optimization restored the files during maintenance.

◆ PAUSE AND DECIDE

What went wrong?

A The deletion was never committed, so the records were never removed.

They were removed. Something put them back.

B The regulations apply to upstream sources too. The records were never deleted from Kafka, so a replay reintroduced them.

Stated directly: deleting inside Databricks is not sufficient. The source still held the records.

C Time travel restored the deleted versions automatically.

Time travel lets you read old versions. It does not write them back.

D Predictive optimization restored the files during maintenance.

Maintenance removes unreferenced files. It never resurrects data.

ON THE EXAM

Three weeks against thirty days of retention is the arithmetic. The records were still in the topic, waiting.

> A team must publish regional health statistics derived from patient records. Some regions have very few patients with a given condition. The published dataset must not be subject to privacy regulation at all.
>
> A colleague suggests hashing the patient identifiers before publishing.

What is the correct approach?

A Hash the identifiers, which removes the personal data from the dataset.

B Suppress the identifying columns and generalise, removing rows where a group is too small to be non-identifying.

C Tokenise the identifiers and keep the vault in a separate catalog.

D Apply a column mask on the identifier column before publishing.

◆ PAUSE AND DECIDE

What is the correct approach?

A Hash the identifiers, which removes the personal data from the dataset.

Hashing is pseudonymisation. The result is still personal data, so the requirement is unmet.

B Suppress the identifying columns and generalise, removing rows where a group is too small to be non-identifying.

Anonymisation, which is what takes the dataset out of scope. The small-group problem is exactly what generalisation addresses.

C Tokenise the identifiers and keep the vault in a separate catalog.

Also pseudonymisation, and a vault makes re-identification easier rather than impossible.

D Apply a column mask on the identifier column before publishing.

Masking is applied at query time on your platform. It does not travel with a published dataset.

ON THE EXAM

The requirement names the test: out of scope entirely. Only anonymisation achieves that, and only anonymisation handles the small-group risk.